PROJECT GUIDE

Autonomous Healthcare-Waste Collection Robot

Explained simply first, then in depth. Written so that every team member can understand, defend and present the project.

IEEE Rwanda Mentorship Program. Team: Damilare Lekan Adekeye, David Parsley Kakhayanga, Beni Bonheur Karangwa. Mentor: Miracle James.

Written 8 November 2026 from the project's own files: the proposal, the team's research on existing robots, the design and test documents in `docs/`, the code in `src/`, and the measured results in `docs/09_Week4_Results.md`. Every number in this guide comes from those files.

How to use this guide

Every important section has three layers. Read as deep as you need.

SIMPLE VERSION: one or two sentences a child could follow.

The proper explanation: the real thing, in plain words, with the exact numbers.

SAY THIS: a ready answer you can say to judges.

CAREFUL: a place where it is easy to overclaim. Read these twice.

If you have 15 minutes: read Part 1, Part 2 (the waste types), Part 5 (the nurse question) and Part 13 (cheat sheet).

If you have one evening: read everything in order, then say the "SAY THIS" boxes out loud.

Before Saturday: everyone should be able to answer the 25 questions in Part 12 without looking.

Contents

1. The project in one minute (the five-year-old version)
2. The waste in a hospital: the types, with examples, and how we use them in the demo
3. What existed before our robot
4. Why a hospital would choose this system
5. The big question: "What if a nurse puts the wrong waste in the wrong bin?"
6. How the robot works, part by part
7. One task, step by step (and what to say during the video)
8. The world we built: rooms, places and the storage bay
9. Maps and remapping: what it is and what it takes
10. How we tested, why, and what the numbers mean
11. Problems we hit and how we fixed them

12. The 25 questions judges may ask
13. Cheat sheet: the numbers and words to remember
14. What is in the repository (a tour of the files)
15. Honest limits and future ideas

Part 1. The project in one minute

SIMPLE VERSION: Imagine a postman robot. But instead of letters, it collects boxes of dangerous hospital waste from rooms and takes them to a safe storage place, so people do not have to carry them. Before it picks a box up, it checks the label on the box. If it is the wrong box, it refuses and goes home.

The proper explanation. A hospital produces waste that can hurt people: used needles, bandages with blood on them, old medicines. That waste is put into special sealed containers in each room. Today, a person must walk to each room, pick up the container, carry it through the corridors, and take it to a storage area. This is repetitive, a little dangerous, and easy to get wrong.

Our robot does that job. Someone sends a request, for example "collect from room A". The robot then:

16. **Drives** to room A by itself, using a map of the clinic.
17. **Looks** at the container with its camera and reads a small black-and-white code (an AprilTag) stuck on it, like a barcode for robots.
18. **Checks** that it is the right container for that room. It needs to see the correct code five times in a row, and no other code.
19. **Picks it up** with a magnet. (In our simulation the magnet is simulated; see Part 6.)
20. **Carries it** to the hazardous-waste storage bay.
21. **Puts it down**, reports "done", and **drives home**.

If the container is wrong or missing, the robot does **not** pick anything up. It reports the reason and drives home.

Everything you see in our video is a **simulation**: a virtual robot in a virtual clinic, running the same kind of software a real robot would run (ROS 2, Nav2, a camera detector). We have not built a physical robot yet.

SAY THIS (30 seconds): "Our project is an autonomous robot that collects sealed healthcare-waste containers from hospital rooms and delivers them to a hazardous-waste store, so staff do not have to carry them. Before it picks anything up, it verifies with its camera that it is the right container, and if anything is wrong it refuses and goes home. We built and tested it in simulation: 67 normal tasks completed out of 67, and 6 out of 6 staged faults refused with the correct cause."

Part 2. The waste in a hospital: the types, with examples

SIMPLE VERSION: Not all hospital rubbish is the same. Some of it is just paper. Some of it can give you a disease, some can cut you, and some can poison you. Each kind goes in its own container.

The proper explanation. The World Health Organization (WHO) groups healthcare waste into categories. The WHO health-care waste fact sheet (who.int/news-room/fact-sheets/detail/health-care-waste, checked 8 November 2026) says that about 85% of the waste produced by health-care activities is general, non-hazardous waste, and the remaining 15% is hazardous: it may be infectious, toxic or radioactive. Two nuances, both from WHO material: other WHO guidance gives a range (75% to 90% comparable to household waste), so say "about 85%"; and where waste is not separated properly, more of it ends up having to be treated as hazardous, so the true hazardous share can be higher. The categories below are the standard ones. Country and hospital rules decide the exact **bin colours**, so do not say "infectious waste is always yellow". Say "colour codes differ by country and hospital; ours follow whatever the hospital's policy says".

Type	What it is	Everyday examples	Why it is dangerous
General (non-hazardous)	Waste with no special risk	Paper, packaging, food waste, office waste	Not dangerous. The big majority of hospital waste
Infectious	Waste that may carry germs	Bandages and dressings with blood, swabs, used gloves, lab cultures, waste from isolation rooms	Can spread infection
Sharps	Anything that can cut or pierce	Needles, syringes, scalpel blades, broken glass vials	Cuts and needle-stick injuries, which can also pass on infection
Pathological (body tissue)	Human tissue	Tissue removed in surgery, placenta, body parts	Infection risk and needs respectful handling
Pharmaceutical	Medicines no longer used	Expired or unused medicines, vials, opened vaccines	Poison, misuse, harm to the environment if thrown away wrongly
Cytotoxic (cancer drugs)	Drugs that damage cells, and what touched them	Chemotherapy drugs, contaminated gloves and tubes	Dangerous to people and the environment even in tiny amounts
Chemical	Chemicals used in cleaning or labs	Lab reagents, solvents, disinfectants, mercury thermometers, batteries	Poisonous, flammable or corrosive
Radioactive	Material from nuclear medicine and radiotherapy	Some diagnostic and treatment materials	Radiation. Needs specially trained handling

What our robot is meant to carry. Our proposal is precise about this: the project "will focus specifically on the transportation of properly segregated and securely sealed healthcare-waste containers. The robot will not sort, open, process, or directly manipulate hazardous waste." So the realistic cargo of a robot like ours is **sealed containers** of infectious waste, sharps boxes (closed), and pharmaceutical waste boxes. Radioactive waste and some cytotoxic waste have special rules, so a robot for those would need extra safeguards; we do not claim to cover them.

How we use the waste types in the demo

Our simulation has three collection containers (tags 11, 12 and 13). The simulation itself only knows their **tag number and colour**; it does not know what is "inside". To make the demo easier to follow, we **tell a story** in which each container holds one type of waste. This is a storytelling choice, not something the software knows, and you should say it that way ("in our demo scenario...").

Room	Container colour on screen	Tag	In our demo story it holds	Why this choice
A	Blue	11	**Infectious waste** (used dressings, swabs, gloves)	The most common stream from a consulting room
B	Green	12	**Sharps** (a closed box of used needles and syringes)	Very familiar to everyone, and the most obviously dangerous to carry by hand
C	Orange	13	**Pharmaceutical waste** (expired or unused medicines)	Shows that "hazardous" is not only about infection

The **storage bay** is "the hazardous-waste temporary store".

CAREFUL: the colours (blue, green, orange) are just the colours we gave the containers on screen so you can tell them apart. They are not an official waste colour code. If a judge asks, say: "the colours here only tell the containers apart; a real hospital would use its own colour code."

Words to say while the video plays

The video is silent, so you narrate it live. This script follows the video's timing (total length 2 min 40 s). Do not read it word for word; use it as a map.

Video time	On screen	Suggest saying
0:00	Title card	"This is a short demonstration, recorded in our simulation of a clinic."
0:06	Card: Demo 1	"In our scenario, a nurse in consulting room A has sealed a container of **infectious waste**: used dressings and swabs. The hospital system calls the robot to collect it."
0:10 to 0:28	Robot drives to room A (fast-forwarded)	"The robot has been called to pick up **infectious waste from room A**. It plans its own path through the corridor. The panel on the right shows each step, and you can see it is on step 1: driving to the collection room."
0:28 to 0:37	Verify and attach (real time)	"At the room it does not trust that it is in the right place. Its camera reads the

		tag on the container. The expected tag, 11, must be seen five frames in a row, with no other tag in view. Then it is verified, and the magnet attaches the container."
0:37 to 1:00	Carrying (fast-forwarded)	"Now it carries the container to the hazardous-waste storage bay. Nobody has touched the infectious waste."
1:00 to 1:16	Release in the bay	"It sets the container down in the marked bay, with the hazard stripes and the shelves, reports 'task completed' and logs the time."
1:16 to 1:24	Drives home	"Then it returns to its home position, ready for the next call."
1:24	Card: Demo 2	"Now the important test: what happens when something is wrong."
1:28 to 1:39	Robot drives to room B	"The robot is called to collect the **sharps container from room B**. But in this test, someone has left the wrong container there: the **pharmaceutical waste box** from room C."
1:39 to 2:08	Verification fails	"Watch the panel. The camera sees tag 13, but for room B it expects tag 12. It tries three times, because a single bad frame should not decide, and then it stops."
2:08 to 2:17	Red banner	"It **refuses**. It picks nothing up. The cause is written to the log: wrong container, tag 13 seen, expected 12."
2:17 to 2:23	Drives home	"And it drives home. The wrong container is not moved."
2:23	Results card	"Over the whole test campaign: 67 normal tasks completed out of 67, and 6 out of 6 staged faults refused with the correct cause."

SAY THIS (the idea behind Demo 2): "In a real hospital, two containers that look alike, or a container put back in the wrong room, is exactly the sort of mix-up that causes incidents. The robot does not rely on assuming it is right; it checks every time."

Part 3. What existed before our robot

SIMPLE VERSION: Before, people did this by hand with trolleys. Some companies already make hospital delivery robots, but they are big commercial products. We are making a small, focused student version that is specially about checking the waste container.

The proper explanation. This section comes from the team's own research document, *Existing Hospital / Indoor Delivery Robots + Task Handling*.

Before robots: the manual way. Cleaners and porters walk to each room, lift the waste containers, push or carry them through corridors, and take them to a storage area, often following a schedule and writing notes on paper. The problems are the four we list in Part 4: handling risk, mix-ups, weak records and tiring repetitive work.

Existing hospital robots. Autonomous mobile robots already exist for hospital logistics:

Robot	What it does	Why it matters to us
Moxi (Diligent Robotics)	Delivers and retrieves medications, lab samples, PPE and supplies. Navigates corridors, avoids people, can use elevators and secure doors where supported	Shows task-based hospital logistics works
TUG and T3 (Aethon)	Transports supplies, medications, meals, linen, and, importantly, **trash and regulated waste**. Can find and connect to a cart	**Our closest baseline**: it proves that hospital waste transport by robot is real and useful
Relay (Relay Robotics)	Point-to-point delivery of specimens, pharmaceuticals, supplies, meals and linen	A clear example of the request, navigate, deliver, complete workflow
Rwanda's COVID-19 robots (Akazuba, Ikirezi, Mwiza, Ngabo, Urumuri) and **UV-C disinfection robots** at Nyarugenge District Hospital	Screening, monitoring, communication, disinfection	Important local context: robots are already accepted in Rwandan hospitals, though these ones do not carry waste

All the delivery robots follow the same pattern, and our software follows it too: a task is created, the robot receives a pickup and a destination, it navigates while avoiding obstacles, it delivers, and it confirms completion.

The gap we aim at. The research found that autonomous indoor logistics and even waste transport are feasible. Our project's focus is the **combination** of waste-container collection, **perception that checks the right container**, indoor navigation and task management, built as a **simulation-first prototype** using open-source software that a student team can afford. Our proposal also says that many existing systems are "expensive, complex, or designed for large commercial facilities", and that there is an opportunity for a smaller, more affordable prototype.

SAY THIS: "Before this, waste moved by hand. Robots like Aethon's TUG already move waste in some hospitals, so we are not claiming to have invented autonomous transport. What we focused on is the check: the robot verifies it has the right container before it picks it up, logs everything, and refuses when something is wrong.

And we built it with open-source tools so a smaller facility could afford to explore it."

CAREFUL: do not say "no one has done this". TUG exists. Say "our focus", not "first".

Part 4. Why a hospital would choose this system

SIMPLE VERSION: It keeps people away from dangerous waste, makes fewer mistakes, remembers everything it did, and does not get tired.

The proper explanation. Five reasons, each tied to something we built or measured.

22. **Safer for staff.** The robot does the carrying. People still seal and place the container, but the long walk with hazardous waste through corridors is done by the machine.
23. **Fewer mix-ups at pickup.** The robot checks the container's tag before lifting it, five frames in a row, and refuses on a mismatch. In our tests all 6 staged wrong-or-missing-container cases were refused with the correct cause.
24. **A clear record.** Every task is logged: time, place, result, how long each stage took, and the cause of any stop. For failures it also saves a camera picture of what it saw. Paper notes depend on people remembering.
25. **Consistent, repeatable work.** The same route and the same checks, every time, at any hour. In 67 logged normal tasks, all completed.
26. **An affordable, adaptable approach.** The software is open source (ROS 2, Nav2, Gazebo, AprilTag), so there are no licence fees. Places are kept in one settings file, so adding a collection room is an edit, and a new layout means re-mapping (Part 9). Because it was built simulation-first, the logic was tested before any hardware was bought.

Honest comparison

	Manual collection	Commercial robot (e.g. TUG)	Our prototype
Staff exposure while carrying	High	Low	Low (in design)
Checks the container before pickup	Depends on the person	Depends on the product	**Yes, camera tag check, refuses on mismatch**
Digital record	Usually paper	Usually yes	Yes, one row per task
Cost	Staff time	Typically high	Aimed at low cost, built on open source
Proven in real hospitals	Yes	Yes	**No: simulation only so far**

SAY THIS: "We would not tell a hospital to replace its process with a student prototype today. What we show is that the design works end to end, that the safety check does its job, and that the next step, a real robot, is realistic because the same software can run on a TurtleBot 4."

CAREFUL: never claim safety certification, clinical validation, or cost savings numbers. We have measured none of those.

Part 5. The big question: "What if a nurse puts the wrong waste in the wrong bin?"

This is the question most likely to be asked, so everyone should be able to answer it calmly. The full question is usually: *"The robot does not put the waste into the container, it only gets called. So what happens when a nurse puts the wrong waste in the wrong bin? What exactly does your robot solve?"*

SIMPLE VERSION: The robot is the delivery person, not the sorter. Sorting happens first, when the nurse fills the bin. Our robot makes sure that the right bin, from the right room, goes to the right place, and that nobody has to carry it.

The proper explanation. Waste handling in a hospital has several layers, and each layer has its own protection:

Step	Who does it	The protection
1. **Sorting at the source**: putting the waste into the right bin	Nurses and staff	Training, colour-coded bins, labels, hospital policy
2. **Sealing and labelling** the container	Nurses and staff	Procedures
3. **Transport** from the room to storage	**Our robot**	**Verified pickup, controlled route, a log, refusal on mismatch**
4. **Storage** until collection	Hospital waste team	Locked, marked hazardous store
5. **Treatment and disposal**	Waste contractors	National regulations

Our proposal states the boundary: *"The robot will not sort, open, process, or directly manipulate hazardous waste."* So:

What our robot does solve (step 3, the transport step):

- Nobody carries hazardous waste by hand through the corridors.
- The wrong container is not picked up: if the container in a room is not the one that was requested, the robot refuses, instead of taking whatever is there.
- A container is never left in the wrong place on the way: it is delivered to the marked store, and each task is logged.
- There is a digital record of what moved, when, and why anything was refused.

What our robot does not do: it does not look inside the bin, so it cannot tell that a nurse dropped a needle into the infectious-waste bin. That is a step-1 problem, handled by staff training and good bin design. We are open about this boundary.

How it connects to the nurse's mistake. If a nurse puts the wrong item in a bin, the bin is still the "infectious waste container with tag 11". The robot then handles it as that container, to the hazardous store, which is a hazardous-waste area anyway. The robot's check protects against the **container** being the wrong one or in the wrong place, which is a different mistake from a wrong item inside it. And because the whole chain is logged, the hospital can trace where a container came from and when it moved.

How the system could go further (future ideas, not promises): a weight check at pickup (a sharps box that is too heavy or light), a lid-closed sensor, a staff "scan to confirm" step when a nurse seals a container, a photo

of each container at pickup, or tags that carry the waste type so the robot can send each type to its own store.

SAY THIS (about 45 seconds): "That is a really important point, and we want to be clear about what our robot does. It does not sort waste. Sorting is done by the nurse when she fills the bin, and that is protected by training and colour-coded bins. Our proposal is deliberately about the next step: transporting sealed, already-sorted containers. In that step, our robot solves three things. People no longer carry hazardous waste through the corridors. The robot checks it has the right container for the room before it lifts it, and refuses if not, so a mix-up at pickup is caught. And every move is logged, so there is a record. If a nurse puts the wrong item in a bin, that is a sorting problem upstream. We could help detect that in future with weight sensors or a scan-to-confirm step, but that is outside what we built."

CAREFUL: do not say the robot "makes sure waste is segregated correctly". It does not. Say "makes sure the right sealed container is moved to the right place".

Part 6. How the robot works, part by part

SIMPLE VERSION: Think of a small company. A manager (the task manager) gives orders. A driver (Nav2) drives. A pair of eyes (the camera and the tag reader) checks the label. A hand (the magnet) picks up the box. They talk by leaving notes on a noticeboard (ROS 2).

6.1 The robot and the world

Name	What it is, in plain words
TurtleBot 4	The robot model we use: a round wheeled robot with a camera and a spinning laser sensor. It is a ready-made, widely used research robot, which is why Nav2's own examples use it and why a real one is a realistic next step
Gazebo	A virtual world with physics (walls are solid, wheels grip). It also simulates what the robot's sensors would see
Simulation	Running the robot's real software inside Gazebo instead of on a real robot. It lets us test safely, repeatedly and for free
ROS 2 Jazzy	The framework that connects the programs (called **nodes**). Nodes talk by publishing and listening to named channels (**topics**), like noticeboards. It is not an operating system; it runs on Ubuntu
LiDAR	A sensor that spins and measures the distance to walls all around, about 10 times a second
Camera	The robot's colour camera, used to read tags (320 x 240 pixels in our simulation)

6.2 Knowing where it is: the map, SLAM and AMCL

SIMPLE VERSION: First the robot draws a floor plan. Later, it finds itself on the plan by looking at the walls around it.

- **Map:** a saved picture of the walls and free space (a `.pgm` image plus a `.yaml` settings file). Ours is `src/hcw_navigation/maps/clinic.yaml`.
- **SLAM** (Simultaneous Localization and Mapping): building the map while moving, using the LiDAR. We do this once, with a tool called SLAM Toolbox.
- **AMCL** (Adaptive Monte Carlo Localization): while working, the robot compares what the LiDAR sees now with the saved map. It keeps many guesses ("particles") about where it is and keeps the ones that fit best, so the guesses collapse to one position. The robot starts at the home spot, and that start pose is given to AMCL.

6.3 Driving: Nav2

SIMPLE VERSION: Nav2 is the driver and the GPS. You say "go to room A" and it works out a route, steers, and

avoids bumping into things.

Nav2 has three parts you should know by name:

- **Planner:** finds a route on the map from where the robot is to the goal.
- **Controller:** steers the wheels along that route and around small obstacles. We use the **MPPI** controller (it tries many possible short paths and picks the best one).
- **Costmap:** a copy of the map that marks danger zones. Walls are expanded by an **inflation** margin (0.35 m in our settings) so the robot keeps a safe distance. The robot's radius is set to 0.18 m.

Settings that matter, and what they mean:

- `xy_goal_tolerance` 0.15 m: the robot counts as arrived when it is within 15 cm of the goal.
- `yaw_goal_tolerance` 0.15 rad (about 8.6 degrees): how much its heading may be off. (We widened this from 0.10 rad; see Part 11.)
- `allow_unknown: false`: it will not plan through parts of the map it has never seen.

Design rule from the mentor: only Nav2 moves the robot. The task manager and the perception code never send wheel commands directly. This makes the system easier to reason about and safer.

6.4 Reading the label: AprilTag and the verification rule

SIMPLE VERSION: An AprilTag is a black-and-white square that a camera can read, like a barcode with a number inside. The robot only believes what it sees if it sees the same number in five pictures in a row.

- Each container carries a tag: **11 for room A, 12 for B, 13 for C**. **14** is a deliberately wrong container used only for tests.
- The tag is 0.30 m wide, because our simulated camera has few pixels.
- The `apriltag_ros` program finds tags in each camera image and reports their numbers.
- Our **verifier** applies the rule: the **expected tag must be seen in 5 consecutive detection messages, and no other tag may be in view.** Only then does it say "verified".

Why five in a row? One image can be wrong: a flicker, a reflection, a half-seen tag. Requiring five in a row, and no other tag, means a single glitch can never trigger a pickup. A pickup is an action you cannot take back, so the system is deliberately strict about it.

What if it is not verified? The task manager waits up to 10 seconds for verification. If that fails, it sends the robot to the same goal again (which realigns it), waits 3 seconds, and tries again, up to 3 attempts. Then it reports FAILED with the cause and goes home. The cause is either "wrong container (tag(s) [13] seen, expected 12)" or "no container tag detected (expected 12)".

This rule is covered by unit tests (Part 10).

6.5 The brain: the task manager (a state machine)

SIMPLE VERSION: It is like a board game. The robot is on exactly one square at a time, and it only moves to the next square when the rule for that square is met.

The task manager runs these states in order:

`IDLE` then `NAVIGATE_TO_POINT` then `VERIFY` then `ATTACH` then `NAVIGATE_TO_STORAGE` then `RELEASE` then `REPORT` then `RETURN_HOME` then `IDLE`.

If anything fails, it goes to `FAILED`: it publishes the cause, puts the container back if one was attached, drives home, and returns to idle. A second request while a task is running is rejected (not queued), and an invalid request such as "Z" is rejected without moving.

It talks to the rest through simple text messages: a request `A`, `B` or `C` goes in; a status message (with the state, the result and the cause) comes out. It writes **one row per task** to a log file (the **CSV**) with time and stage timings.

6.6 The magnet (simulated)

SIMPLE VERSION: In the real world a magnet would grab the box. In our video, the box is made to float along with the robot, as if it were stuck on by a magnet.

In the simulation, "attach" means the container model follows the robot while carried (it floats about 0.45 m above the robot), and "release" means it is put down at the storage drop point. We describe it as a **simulated magnetic latch**. It was chosen because building a real gripper in a simulator is slow and unreliable, and because the project's main questions are about navigation, verification and task logic.

CAREFUL: say "simulated magnet" every time. Never imply a real gripper was built or tested.

6.7 How the parts talk

```
request "A" ---> TASK MANAGER ---> Nav2 (drives)
                    |  ^
       expected tag |  | verified / not verified
                    v  |
                VERIFIER <--- AprilTag detections <--- camera
                    |
                    +--> MAGNET (attach / release)
                    +--> status message and log file
```

Part 7. One task, step by step

This is what happens when someone asks for room A, and what each stage of the on-screen panel means.

Step	Panel says	What really happens	Typical time
1	Drive to the collection room	Nav2 plans a path to the point in the corridor in front of the room (13.6, -18.6) and drives there	about 33 s on average (A about 39 s, B about 24 s, C about 13 s)
2	Verify the container tag	Tag 11 must be seen in 5 frames in a row and no other tag. The robot stops 1.8 m from the container, which stands inside the room, tag facing the corridor	about 0.5 s when correct
3	Attach with the magnet	The simulated magnet latches the container	instant
4	Carry to the storage bay	Nav2 drives to the storage point (30.9, -11.5)	about 56 s
5	Release at storage	The container is set down against the north wall (30.9, -10.95)	instant
6	Report completion	The task manager publishes COMPLETED with the timings and logs a row	instant
7	Drive back home	Nav2 returns to the start spot (8.0, -31.7)	about 1 to 2 minutes

From request to REPORT: mean 89 s, median 87 s over the 30-task final batch. With the drive home, a task takes about 3.3 minutes.

Failure branch: at step 2, a wrong or missing container ends the task. The panel shows the red cross on "Verify the container tag", the cause, and the robot drives home. It picks nothing up.

SAY THIS: "A task is seven steps. The most important one is step two. Everything before it is just driving; step two is where the robot decides whether it is allowed to pick up."

Part 8. The world we built

SIMPLE VERSION: We took a real hospital design, the ground floor of an open-source clinic, and put our robot, three waste rooms and a hazardous-waste bay inside it.

The proper explanation. The clinic model comes from Open-RMF's clinic world (an open-source robotics project). We patched it for our purposes: removed the lifts and automatic doors that need a special plugin (they otherwise block doorways), removed the other robots, and made decorative parts static so the simulator stays fast.

Named places (coordinates in metres, in the same frame as the map). These are in `src/hcw_world/config/locations.yaml`:

Place	x	y	Meaning
home	8.0	-31.7	Where the robot starts and returns
collection A	13.6	-18.6	Corridor in front of consulting room 12. Tag 11 container 1.8 m ahead
collection B	13.6	-26.2	Consulting room 10. Tag 12
collection C	13.6	-33.0	Consulting room 8. Tag 13
storage	30.9	-11.5	Storage bay in the north corridor (robot stops here)
drop point	30.9	-10.95	Where the container is set down, against the north wall

The storage bay. In demo mode the storage area is dressed as a hazardous-waste store: an amber hazard-striped floor with biohazard-style symbols, shelves with warning-coloured cartons, yellow clinical-waste bins, a red sharps box, corner posts and two wall signs reading "HAZARDOUS WASTE, AUTHORISED ACCESS ONLY". It is a marked area in the north corridor.

If a judge asks why it is in the corridor and not a separate room, the true answer is a good engineering story:

SAY THIS (only if asked): "The clinic does have a store room in the north-east hall. When we mapped that large open hall, the SLAM map drifted: open space gives the laser few walls to measure against. Nav2 could not plan reliably there, and in four repeated runs the trip home from that area failed. So we put the storage bay in the well-mapped north corridor, and made it look and work like a hazardous-waste store. A dedicated room is a clear next step: it needs that area re-mapped and the settings re-tuned."

Hospital look. For the demo we add beds, desks and furniture to the rooms (props). These cost almost nothing on a free laptop but about 10% speed on a busy one, so they are an optional mode.

Part 9. Maps and remapping

This part is for the question: *"What if we want to move the robot from this side, from the corridor, to a room across the hospital? What do we do?"* The key words are **remapping**, **localisation**, **named places** and **re-tuning**.

SIMPLE VERSION: The map is the robot's floor plan. If the building changes or the robot has to go somewhere the plan does not show, you walk the robot there once to draw the missing part, then you tell it the new place names.

9.1 Four cases, from easy to hard

Case	Example	Do we need to remap?	What we do	Effort
1. New place inside the already-mapped area	Add a fourth collection room next to room B	**No**	Add the place and its tag in `locations.yaml`, put the container, test with `go_to`	Minutes
2. A part of the same building not yet mapped	A room "across the hospital" in the east wing	**Yes, for that area**	Map the area (below), update the place list, re-test	A few hours
3. Another floor	First floor	**Yes, a separate map for each floor**	New map per floor, plus a way to use the lift (not built)	Large (future)
4. A different hospital	Another clinic	**Yes, fully**	New map, new place list, new tags. The software stays the same	A day or more per building

In our own project: the west wing, the north corridor and part of the north-east hall are mapped. The east wing and the middle of the building are not. Case 2 is exactly "the room across the hospital".

9.2 What remapping involves, step by step

This is how we made our map, and how a new area would be added (from `docs/05_Mapping_Navigation_Perception.md`):

27. **Start mapping mode:** `ros2 launch hcw_navigation mapping.launch.py` starts SLAM Toolbox.
28. **Drive the robot through the area.** In our clinic, `ros2 run hcw_navigation map_clinic` drives automatically along the building's own corridor graph, in steps of 3 m (Nav2 refuses goals outside the part of the map built so far, so steps matter). In a new building with no such graph, you drive with the arrow-key teleop or a joystick while SLAM builds the map. This takes minutes to an hour per area.
29. **Save the map** with the map-saver tool, and shift its origin so the map's coordinates equal the world's coordinates.
30. **Clean the map:** we removed speckle (stray scan points shorter than 1.5 m in the open hall).

31. **Seal doorways where needed:** the insides of our rooms were unmapped, and Nav2 took shortcuts into them and could not plan from inside. `seal_room_doors.py` closes the doorways on the map; the robot stops in the corridor and the camera sees the container through the open door.
32. **Check alignment:** `plot_map.py` draws the named places on the map. They must fall on corridors and wall openings.
33. **Update the places:** add each new place (x, y, heading, and tag number) to `locations.yaml`.
34. **Place tags and containers**, then **test:** `scripts/test_all_places.sh` drives to every place with tag checks (about 6 minutes), then run a few full tasks.
35. **Re-tune if needed:** if the new area has different corridor widths, the Nav2 margins (inflation, tolerances) may need adjusting.

9.3 What makes remapping hard (and what we learned)

- **Large open spaces** give the laser few walls, so the map drifts. Our north-east hall was the hard part.
- **Changes in the building** (moved furniture, new walls) make the map out of date, so maps are refreshed from time to time.
- **People and moving objects** are not drawn on the map; Nav2 handles them live with the laser. We did not test moving people in our simulation.
- **Doors and lifts** must be handled: our simulation removed them, and a real hospital would need the robot to open or request them (Moxi does this "where supported").

SAY THIS: "Remapping means rebuilding the robot's floor plan for a new area. If the room across the hospital is in a part we have not mapped, we drive the robot there once while SLAM builds the map, save it, clean it, add the new place to our locations file, and test. If it is in an area we have already mapped, we do not remap at all; we just add a new named place. The robot's software does not change; only the map and the list of places do."

Part 10. How we tested, why, and what the numbers mean

SIMPLE VERSION: We did not just try it once. We made the robot do the job again and again, we also tried to trick it, and we checked it again after every change.

10.1 Types of testing we used, and why

Type	What it means	What we did	Why
Unit test	Test one small piece alone	8 automated tests of the verification rule (`hcw_perception/test/test_verifier_rule.py`)	The rule decides when it is safe to pick up. Some cases, like a flickering tag, cannot be produced reliably in the simulator, so we test the rule directly
Integration test	Test that parts work together	`test_all_places.sh`: drive home, A, B, C, storage, home with tag checks. Result: 15 of 15 legs, correct tag detected 9 of 9 times	Navigation and perception must agree before we test tasks
System (end-to-end) test	Test the whole job	Full tasks to A, B and C (test cases TC01 to TC03)	This is what a user would see
Fault-injection (negative) tests	Deliberately cause problems	Wrong container (TC04, 05, 05b), missing container (TC06, 07, 07b), corridor blocked (TC10), busy request (TC12), invalid request (TC13)	A safety system is judged by how it fails, not only by how it succeeds
Regression test	Re-test after a change	Re-checks on 7 and 8 November after code changes	A fix in one place must not break another
Repeatability batches	Many runs, unattended	`run_batch.sh`: A, B and C in rotation, plus the fault cases	One success proves little; many in a row show consistency
Performance check	Is the simulation fast enough?	`check_speed.sh` (real-time factor)	A slow simulator changes robot behaviour and hides problems
Root-cause investigation	Find why something failed	Saved camera pictures at failures; logged the robot's pose every 0.5 s	A fix must come from evidence, not guesswork

Why simulation-first testing? It is safe (no real waste, no real hospital), repeatable (the same conditions again and again), free, and fast, and it works with the hardware we could access. The cost is that simulation is not the real world, which is why we keep saying "simulation".

10.2 The results

Measure	Result
Normal tasks completed (final settings)	**67 of 67** (A 37, B 15, C 15)
Staged faults refused with the correct cause	**6 of 6** (missing and wrong container at each of A, B, C)
Time from request to REPORT	mean 89 s, median 87 s, min 85 s, max 146 s
Drive to the collection point	about 33 s on average; transport to storage about 56 s
A whole task including the drive home	about 3.3 minutes

The history shows the engineering. Under looser settings (runs 20 to 55) there were 3 failures in 30 normal tasks, all at room A: the robot stopped about 18 degrees off its heading, so the tag sat at the edge of the camera picture and could not be read. We found that by saving the camera frame at each failure. We tightened the settings and added a realign-and-retry step, and then 12 of 12 runs at A, and later more, completed. (Details: `docs/09_Week4_Results.md`, section 2.)

10.3 What "67 of 67" really means (and why it is "few")

First, an important correction to a common idea: **nothing in this project was "trained".** There is no machine learning, no training set. The robot uses:

- **Planning and control algorithms** (Nav2), which follow mathematical rules.
- **A saved map** and a localisation method (AMCL).
- **AprilTag detection**, which is classical computer vision (it finds a known square pattern), not a learned model.

So **the 67 tasks are test runs, not training data.** The closest thing to "training" in our project is **tuning**: changing a setting after a test showed a problem. Example: the heading tolerance was raised from 0.10 to 0.15 rad after logs showed the robot stalling about 1 degree outside the old limit.

Why we call the number "few" for engineering purposes. 67 sounds like a lot, because each run takes minutes. But for judging reliability:

- A fault that happens, say, 1 time in 100 could easily not appear in 67 runs.
- Statistics gives the limit: if all 67 runs succeed, we can say, with 95% confidence, that the true success rate is **at least about 96%**. We cannot say "100%" or "never fails". (A quick rule: with 0 failures in n runs, the failure rate is probably below 3 divided by n; here 3/67 is about 4.5%.)
- Real hospital robots are tested over thousands of tasks, in many conditions: different lighting, people walking, different times, hardware wear.
- Our 67 runs also share narrow conditions: one world, one start position, one machine, containers always in the same place, fixed lighting.

So the right way to say it is: 67 of 67 is **good prototype evidence that the design works**. It is not proof of real-world reliability. Saying that out loud makes you look more credible, not less.

SAY THIS: "We ran 67 normal tasks and all 67 completed, plus six staged faults and the robot refused all six correctly. We're careful about what that means: nothing here was machine-learned, these are test runs, and with 67 runs we can say the success rate is at least about 96% with 95% confidence. That is strong evidence for

a simulation prototype. A real deployment would need thousands of runs in real conditions."

Not yet tested, and we say so: the robot losing its route while carrying a container (test TC11), a magnet failure at attach or release (TC14, TC15), moving people, and more than one task at a time. The unstable-tag case is covered by the unit tests, not by a live run.

Part 11. Problems we hit and how we fixed them

These stories show real engineering. Know them well.

36. **The doors blocked the robot.** The clinic world has about 140 door and lift models that only open with a special plugin. Without it they stayed shut. *Fix:* remove every model that has a joint.
37. **The simulation was too slow (1 to 3% of real time).** Hundreds of detailed models made the camera and laser slow. *Fix:* a simplified "building" version of the world, and only simple shapes for props. Now about 100% on a free laptop.
38. **The robot drove into unmapped space.** Nav2 plans through unknown areas by default. *Fix:* `allow_unknown: false`.
39. **The robot reset its own position to zero.** A start-up function quietly reset the localisation to the origin. *Fix:* start it in a way that does not reset.
40. **The tag looked like a white square.** A glowing material washed out the pattern. *Fix:* correct texture and a bigger tag (0.30 m), since the camera has few pixels.
41. **Failures at room A.** The robot stopped about 18 degrees off, the tag was at the edge of the picture. *Fix:* tighter settings plus a realign-and-retry step (Part 10.2).
42. **Slow trips to room A (84 to 98 seconds instead of about 39).** We logged the robot's position every 0.5 s and found it stopped 0.02 rad (about 1 degree) outside the heading tolerance, so Nav2 waited for its "failed to make progress" timer, four times, until a recovery spin fixed it. *Fix:* heading tolerance 0.10 to 0.15 rad. Afterwards trips to A took about 39 s again.
43. **A leftover program froze the batches.** An old task manager ignored the stop signal. *Fix:* the cleanup script now force-stops our own programs and the start script refuses to start if one remains.
44. **Nav2 sometimes does not start (about 1 in 3 starts when the laptop is busy).** *Fix:* the start script waits until Nav2's main component is active and retries up to 3 times. Each retry costs about 3 minutes.
45. **The container was dropped in the middle of the hallway.** It looked wrong and unsafe. *Fix:* the drop point moved against the wall, and a marked hazardous-waste bay was built there.

SAY THIS (a good story for any "what went wrong" question): "At room A the robot sometimes took 90 seconds instead of 40. We did not guess. We logged its position every half second and found it was stopping one degree outside its turning tolerance, so the navigation system waited for a timeout before correcting itself. We widened the tolerance slightly and the trips went back to 39 seconds. That is how we worked on everything: measure, find the cause, fix, and test again."

Part 12. The 25 questions judges may ask

Short, honest answers. Learn the idea, not the exact words.

1. What problem does your robot solve?

Four: risky manual handling of hazardous waste, mix-ups of containers, weak records of what moved, and tiring repeated carrying. It does the carrying, checks the container before pickup, and logs every task.

2. What existed before your robot?

Manual collection by staff with trolleys, and commercial hospital robots such as Aethon TUG (which can carry trash and regulated waste), Moxi and Relay for supplies and medicines. Rwanda has used robots for screening and UV disinfection. Our focus is a small, affordable, open-source prototype with a container check.

3. Why should hospitals choose your system?

Safer for staff, fewer mix-ups at pickup, a digital record, consistent work, and an open-source approach that is adaptable. We are honest that it is a simulation prototype today.

4. What if a nurse puts the wrong waste in the wrong bin?

That is a sorting problem upstream, solved by training and colour-coded bins. Our robot transports sealed, already-sorted containers and makes sure the right one goes to the right place. See Part 5.

5. What exactly does your robot solve?

The transport step: no hand-carrying of hazardous waste, a check before pickup, a refusal on mismatch, and a log.

6. Is it a real robot?

Not yet. It is a simulation using the same software stack a real TurtleBot 4 would run. The next step is real hardware.

7. Why simulation first?

Safe, repeatable, free, and fast; it let us test the software logic thoroughly before buying hardware.

8. How does the robot know where it is?

A map built with SLAM, and AMCL localisation that compares the laser readings with the map.

9. How does it avoid obstacles?

Nav2 uses the laser to see obstacles and drives around them. We tested a box that left a gap (it drove around) and a fully blocked corridor (it failed cleanly after about 64 seconds and went home). We did not test moving people.

10. How does it know it has the right container?

It reads the container's AprilTag with the camera. The expected number must be seen five frames in a row and no other tag may be in view.

11. Why five frames?

One frame can be wrong. Five in a row avoids a pickup caused by a flicker or reflection.

12. What if the tag is dirty, damaged or hidden?

Then no tag is verified, so the robot does not pick up and reports "no container tag detected". It fails safe. We did not test dirty or damaged tags.

13. Why AprilTag and not RFID or QR?

AprilTags are made to be read reliably by robot cameras at a distance and at angles, they are free to print, and they work with ROS. The proposal also mentioned RFID as an option for later.

14. Does the robot open or sort the waste?

No. It never opens, sorts or processes waste; it moves sealed containers.

15. What is the magnet?

A simulated magnetic latch: in the simulation the container follows the robot while attached. A real robot would need a real gripper or latch; that is future work.

16. What happens if something goes wrong while it is carrying a container?

By design, the task manager puts the container back at its original place and drives home. We tested the failure cases at the collection point, but not a navigation failure while carrying. We list it as untested.

17. How many tests did you do, and is that enough?

67 normal tasks, all completed, and 6 staged faults, all refused correctly. That supports at least about 96% success with 95% confidence. It is good prototype evidence; real deployment needs thousands of runs in real conditions.

18. Did you train a model? What data did you use?

No machine learning. Navigation uses planning algorithms and a map, detection uses AprilTags (classical vision). The 67 runs are test runs. The closest thing to training is tuning settings, for example the heading tolerance.

19. What is remapping, and what would it take to add a room across the hospital?

Remapping is rebuilding the robot's floor plan for a new area. In an already-mapped area, we only add a named place. In an unmapped area, we drive the robot through it while SLAM builds the map, save and clean the map, add the place and tag, and test. The software does not change. See Part 9.

20. How long does one task take?

About 89 seconds from request to completion report, and about 3.3 minutes including the drive home, in simulation.

21. How accurate is it?

In our 24 October reliability run, the final position error at the named places was 0.02 to 0.31 m (the goal tolerance then was 0.25 m; it is 0.15 m now). Accuracy on a real robot would need to be measured again.

22. Why a corridor storage bay and not a separate room?

The room in the north-east hall was poorly mapped, because open halls make the SLAM map drift. The bay in the corridor is well mapped. A dedicated room is a clear next step.

23. What about safety around people?

Nav2 uses the laser to detect obstacles and plan around them. We did not test moving people in the simulation, and a real deployment would need proper safety certification and an emergency stop. We do not claim it is safe in a real hospital yet.

24. What are the limits, and what is next?

Simulation only, a simulated magnet, narrow test conditions. Next: a real TurtleBot 4 with a real gripper, many more tests including failures while carrying, a request screen, alerts, more rooms and floors, and several robots working together.

25. What did each of you do, and which tools did you use?

Prepare your own two-sentence answer about your personal contribution. Be truthful about tools: the repository's `docs/TEAM.md` already says the simulation, navigation, perception and task-manager code was built on Dami's laptop with an AI coding assistant (Claude Code) doing much of the typing, running and measuring, under the team's direction, with the team making the decisions recorded in `DECISIONS.md`. If a judge asks about tools, answer plainly and focus on what you decided, tested and learned. Whatever you say should match the repository.

Part 13. Cheat sheet: the numbers and words

Numbers

Item	Value
Normal tasks	**67 of 67** completed (A 37, B 15, C 15)
Staged faults	**6 of 6** refused with the correct cause
Request to report	mean 89 s, median 87 s
Whole task with drive home	about 3.3 min
Confidence statement	at least about 96% success, with 95% confidence
Tags	11 = A, 12 = B, 13 = C, 14 = wrong container (test only)
Verification rule	expected tag in 5 consecutive frames and no other tag
Verify timing	up to 10 s per attempt, 3 attempts, 3 s between
Goal tolerances	0.15 m position, 0.15 rad heading
Inflation / robot radius	0.35 m / 0.18 m
Tag size	0.30 m
Robot stop distance from container	1.8 m
Slow-trip fix	heading tolerance 0.10 to 0.15 rad; trips to A back to about 39 s
Simulation speed	about 100% of real time on a free laptop

Keywords to say with confidence

Word	In one line
ROS 2	Framework that connects the robot's programs (nodes) with topics
Gazebo	The virtual world used for simulation
Simulation-first	Test the full software in a virtual world before any hardware
TurtleBot 4	The robot model: round, wheeled, camera and laser
LiDAR	Spinning laser distance sensor
SLAM	Building a map while moving
Map	Saved floor-plan image of walls and free space
Remapping	Rebuilding or extending the map for a new or changed area
AMCL	Finds the robot's position on the map
Nav2	The driving system: planner, controller, costmap
Costmap / inflation	Safety margin around walls
Tolerance	How close or how aligned counts as "arrived"
AprilTag	Robot-readable square marker with an ID
Verification rule	5 frames in a row, expected tag only
State machine	The task manager: one stage at a time
Fault injection	Deliberately causing a failure to test the response
Regression test	Re-testing after a change
Root cause	The real reason for a failure, found with evidence
Segregation	Sorting waste into the right category at the source
Fail safe	When unsure, do nothing dangerous, stop and report

Phrases that make you sound credible

- "In simulation, we measured..."
- "We deliberately limited the scope to sealed, already-sorted containers."
- "Here is what we did not test yet."
- "We found the cause by logging, then fixed it, then re-tested."

Phrases to avoid

- "It never fails." / "100% safe." / "Fully autonomous in hospitals."
- "The robot sorts the waste."
- "We trained the robot with 67 examples."
- "No one has done this before."

Part 14. What is in the repository

The repository is a ROS 2 workspace. Everything runs on Dami's laptop (WSL2 Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic). Teammates can read all of it without running it.

Folder or file	What it is
`src/hcw_world/`	The simulated world. `world_builder.py` patches the clinic world; `config/locations.yaml` has every named place and container; `models/` has the containers and tags; `scripts/` makes tag models and hazard textures
`src/hcw_bringup/`	Starts the simulation. `launch/sim_clinic.launch.py`, and `teleop_arrows.py` (drive with arrow keys)
`src/hcw_navigation/`	Navigation. `config/nav2_params.yaml` (all the tolerances), `maps/clinic.yaml` and `.pgm` (the map), `launch/` (navigation and mapping), `go_to.py`, `map_clinic.py`, `scripts/seal_room_doors.py`, `scripts/plot_map.py`
`src/hcw_perception/`	Tag reading. `verifier.py` (the 5-frame rule), `tag_monitor.py`, `config/apriltag.yaml`, `test/test_verifier_rule.py` (unit tests)
`src/hcw_task/`	The task manager. `task_manager.py` (state machine and log), `magnet.py` (simulated magnet), `request.py` (send a request)
`scripts/`	`env.sh`, `clean_sims.sh`, `start_stack.sh`, `run_batch.sh`, `summarize_runs.py`, `check_speed.sh`, `test_all_places.sh`, and the demo helpers (`demo_camera.sh`, `demo_fault.sh`, and others)
`tools/`	Demo tools: the on-screen status panel, the video builder, the slide builder
`docs/`	`00` to `11`: quick start, setup, evaluations, bring-up, mapping, task manager design, weekly results; plus `COMMANDS.md`, `GLOSSARY.md`, `DECISIONS.md`, `TEAM.md`, and `docs/results/` (test plan and run data)
`CLAUDE.md`, `CONTRIBUTING.md`	Working rules for the repository

The files to read first, in order: `docs/GLOSSARY.md`, `docs/07_Task_Manager_Design.md`, `src/hcw_world/config/locations.yaml`, `docs/09_Week4_Results.md`, `docs/results/test_plan.md`, `docs/DECISIONS.md`.

To run the simulation (on Dami's laptop): close heavy programs, then run `bash scripts/start_stack.sh gui props` (about 1.5 minutes; if Nav2 does not start, it retries, about 3 minutes each time), then `ros2 run hcw_task request A`. `bash scripts/check_speed.sh` tells you if the laptop is fast enough. `docs/COMMANDS.md` lists every command.

CAREFUL: if a judge asks you to run it live, decide in advance. A cold start can take 2 to 8 minutes. The video is the safe option; offer a live run only if there is time.

Part 15. Honest limits and future ideas

What we have not done. We built and tested this in simulation. The magnet is simulated. Test conditions were narrow: one world, one start position, fixed lighting, containers in fixed places. Some tests were not run live: a route failure while carrying, a magnet failure, moving people, or several tasks at once. Starting Nav2 occasionally needs a retry on our laptop.

Why saying so helps you. Judges expect a student prototype to have limits. The teams that look strong are the ones who can state exactly where theirs are. Everything in this section can be said in a calm, confident voice.

Ideas for what we could work on next (ideas, not promises):

46. **A real robot:** run the same software on a real TurtleBot 4 with a real magnet or gripper.
47. **More testing:** many more tasks, long shifts, failures while carrying, and different lighting.
48. **A request screen:** a simple web page or app so staff can call the robot in one tap.
49. **Alerts and a dashboard:** live status, and a message to staff when a task is refused.
50. **More rooms and floors:** more collection points and lifts for several floors.
51. **A team of robots:** several robots sharing the work, as in the open-source Open-RMF fleet system the clinic world comes from.
52. **Smarter checks:** weight check, lid-closed sensor, scan-to-confirm, tags carrying the waste type with a store for each type.
53. **A dedicated store room:** re-map the north-east hall and tune the settings.

End of guide. Update this file as the project changes. The copy in `docs/11_Project_Guide.md` is the master; Word and PDF copies are for reading on phones.